Name - Chaitanya Kelkar

Roll no. - 53

Experiment 6

Aim - Apply K-Means and Hierarchical Clustering on sample datasets.

2. Dataset Source & Description

- **Source:** Kaggle (Crop_recommendation.csv)
- **Description:** Multivariate agricultural data. For this unsupervised learning experiment, the target variable (`label`) is intentionally removed.
- **Size:** 2200 instances, 7 continuous features.
- **Features:** N, P, K (Soil Nutrients) and temperature, humidity, ph, rainfall (Environmental Conditions).

3. Mathematical Formulation of the Algorithm

- **A. K-Means Clustering:** The goal is to divide data into K distinct clusters by minimizing the variance within each cluster, known as the Within-Cluster Sum of Squares (WCSS). $WCSS = \Sigma \|x_i - C_j\|^2$ *(Where x_i is a data point and C_j is the centroid of its assigned cluster)*
- **B. Hierarchical Clustering (Agglomerative):** A bottom-up approach where every data point starts as its own cluster. The algorithm iteratively merges the two closest clusters based on a distance metric until only one massive cluster remains. We use **Ward's Method**, which merges clusters in a way that minimizes the total variance within all clusters.

4. Algorithm Limitations

- **K-Means:** Requires the user to manually guess or calculate the number of clusters (K) beforehand. It assumes clusters are spherical and is highly sensitive to outliers.
- **Hierarchical Clustering:** Extremely computationally expensive ($O(N^3)$). Once two clusters are merged, the algorithm cannot undo that step, meaning early mistakes are permanent. It is not suitable for massive datasets.

5. Methodology / Workflow The experiment followed an Unsupervised Learning pipeline:

- **Data Preprocessing:** Dropped the `label` column entirely.
- **Standardization:** Applied StandardScaler. Since both algorithms calculate physical distances between data points, scaling is mandatory so features with large numbers (Rainfall) don't overpower smaller ones (pH).

- **K-Means Optimization:** Ran a loop testing K=1 to K=10, recording the WCSS to plot the **Elbow Curve**.
- **Hierarchical Visualization:** Built a **Dendrogram** using the SciPy library to visually trace the hierarchy of cluster merging.
- **Dimensionality Reduction:** Used Principal Component Analysis (PCA) to compress the 7 dimensions into 2D so the final clusters could be visualized on a scatter plot.

6. Performance Analysis

- **Elbow Method:** The WCSS graph drops rapidly and then creates an "elbow" shape. The point where the drop stabilizes indicates the optimal number of natural clusters in the data.
- **Dendrogram Analysis:** The optimal number of clusters is found by drawing a horizontal line across the longest vertical distance that doesn't intersect any other cluster branches.
- **Cluster Characteristics:** The algorithms successfully grouped the data into macro-categories (e.g., High-Rainfall/High-Humidity crops vs. Arid/Low-Water crops).

7. Hyperparameter Tuning

- **K-Means (n_clusters):** Tuned using the Elbow Method to find the most mathematically sound number of groupings.
- **K-Means (init):** Set to 'k-means++' to intelligently place initial centroids, avoiding the "random initialization trap."
- **Hierarchical (linkage):** Tuned to 'ward' to minimize variance, and the distance metric was set to 'euclidean'.

Code -

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import KMeans, AgglomerativeClustering

from sklearn.decomposition import PCA

import scipy.cluster.hierarchy as sch
```

```python
from sklearn.metrics import silhouette_score


# 1. DATA LOADING & PREPROCESSING

print("Loading Data...")

try:

    df = pd.read_csv('./data/Crop_recommendation.csv') # Adjust path if
needed

except FileNotFoundError:

    print("Error: File not found. Check your data path.")

    exit()


# For UNSUPERVISED learning, we MUST drop the target label

X = df[['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']]


# Feature Scaling (Crucial for distance-based clustering)

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)


print("Data scaled and ready for clustering.")


# 2. K-MEANS & THE ELBOW METHOD

print("\n--- Running K-Means Elbow Method ---")

wcss = []

K_RANGE = range(1, 11) # Testing 1 to 10 clusters
```

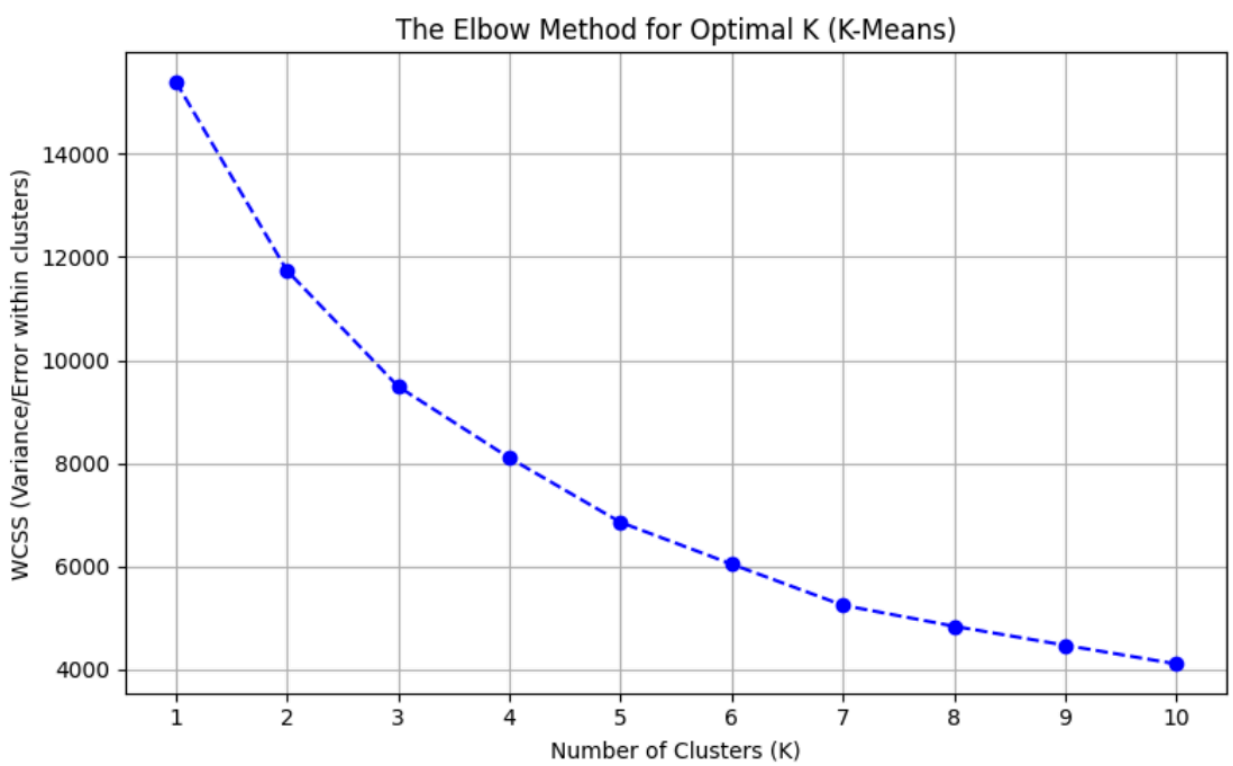
```
for i in K_RANGE:
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42,
n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

# Plot the Elbow Method Graph
plt.figure(figsize=(8, 5))
plt.plot(K_RANGE, wcss, marker='o', linestyle='--', color='b')
plt.title('The Elbow Method for Optimal K (K-Means)')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS (Variance/Error within clusters)')
plt.xticks(K_RANGE)
plt.grid(True)
plt.tight_layout()
plt.savefig("exp6_kmeans_elbow.png")
print("Saved 'exp6_kmeans_elbow.png'")

# Decide on an optimal K based on standard agricultural groupings (e.g., 4
macro-clusters)
OPTIMAL_K = 4

# Train final K-Means model
```

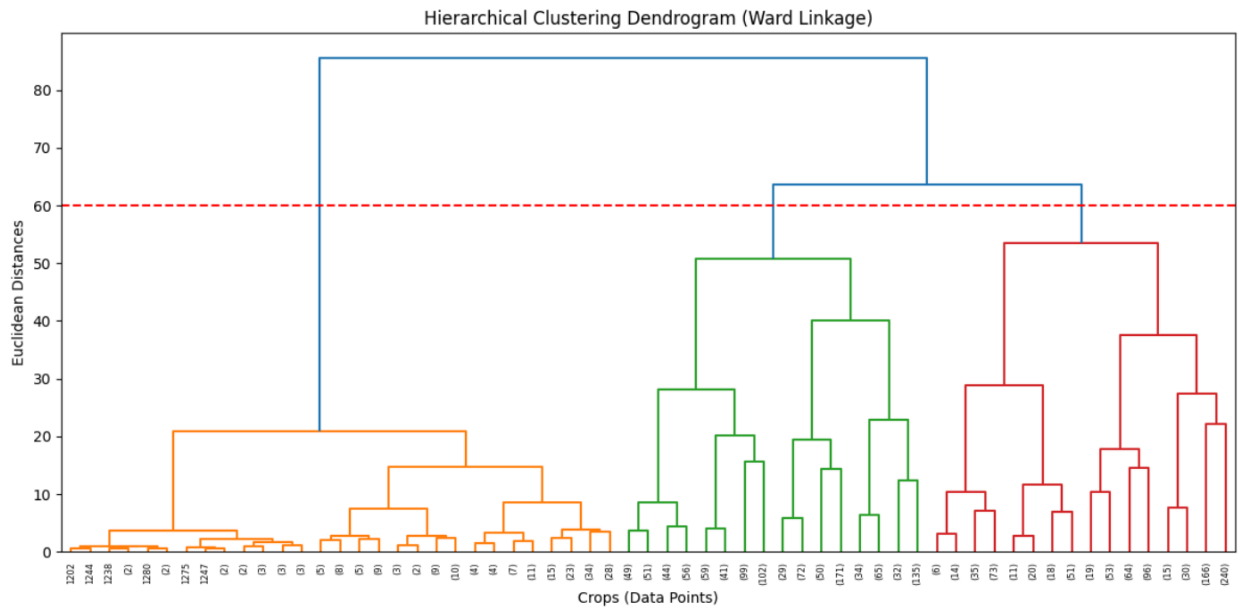
```
kmeans_final = KMeans(n_clusters=OPTIMAL_K, init='k-means++',
random_state=42, n_init=10)

kmeans_clusters = kmeans_final.fit_predict(X_scaled)



# Calculate Silhouette Score (closer to 1 is better)

sil_score = silhouette_score(X_scaled, kmeans_clusters)

print(f"K-Means Silhouette Score (K={OPTIMAL_K}): {sil_score:.3f}")



# 3. HIERARCHICAL CLUSTERING & DENDROGRAM

print("\n--- Generating Hierarchical Dendrogram ---")

# Warning: Dendrograms on 2200 rows can be visually dense!

plt.figure(figsize=(12, 6))

plt.title('Hierarchical Clustering Dendrogram (Ward Linkage)')

plt.xlabel('Crops (Data Points)')

plt.ylabel('Euclidean Distances')



# Generate the linkage matrix

linkage_matrix = sch.linkage(X_scaled, method='ward')



# Plot the dendrogram (we truncate it to show the top levels cleanly)

sch.dendrogram(linkage_matrix, truncate_mode='level', p=5,
show_leaf_counts=True)

plt.axhline(y=60, color='r', linestyle='--') # Horizontal line cutting the
longest branches
```

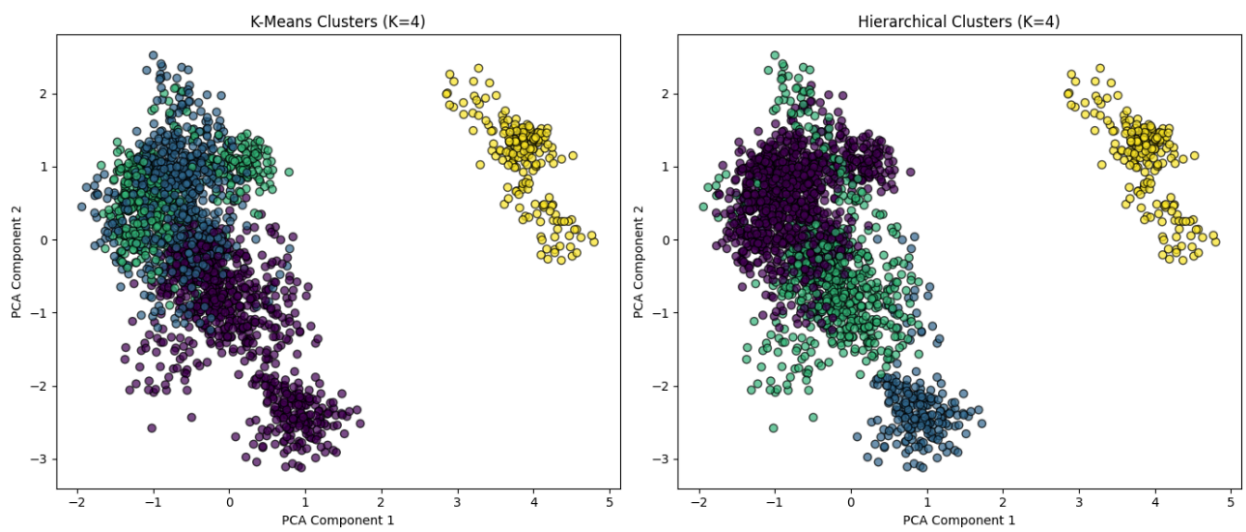
```
plt.tight_layout()

plt.savefig("exp6_hierarchical_dendrogram.png")

print("Saved 'exp6_hierarchical_dendrogram.png'")


# Train final Hierarchical model

hc = AgglomerativeClustering(n_clusters=OPTIMAL_K, metric='euclidean',
linkage='ward')

hc_clusters = hc.fit_predict(X_scaled)


# 4. 2D CLUSTER VISUALIZATION (PCA)

print("\n--- Visualizing Final Clusters via PCA ---")

# Compress 7 features into 2 dimensions for graphing

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)


# Create a visual comparison of K-Means vs Hierarchical side-by-side

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))


# Plot K-Means Clusters

scatter1 = ax1.scatter(X_pca[:, 0], X_pca[:, 1], c=kmeans_clusters,
cmap='viridis', edgecolor='k', s=40, alpha=0.7)

ax1.set_title(f'K-Means Clusters (K={OPTIMAL_K})')

ax1.set_xlabel('PCA Component 1')

ax1.set_ylabel('PCA Component 2')
```

```
# Plot Hierarchical Clusters

scatter2 = ax2.scatter(X_pca[:, 0], X_pca[:, 1], c=hc_clusters,
cmap='viridis', edgecolor='k', s=40, alpha=0.7)

ax2.set_title(f'Hierarchical Clusters (K={OPTIMAL_K})')

ax2.set_xlabel('PCA Component 1')

ax2.set_ylabel('PCA Component 2')



plt.tight_layout()

plt.savefig("exp6_cluster_comparison_2d.png")

print("Saved 'exp6_cluster_comparison_2d.png'. Experiment Complete!")
```

Output -







Conclusion -

This experiment successfully applied unsupervised machine learning (K-Means and Hierarchical Clustering) to identify natural groupings in agricultural data based purely on soil and weather features. The optimal number of clusters was determined using the Elbow Method and a Dendrogram using Ward's Linkage, and the final groups were clearly visualized using PCA. Ultimately, the results demonstrate that clustering can effectively categorize crops into distinct environmental profiles, providing valuable insights for alternative crop recommendations without needing predefined labels.